MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Implementation of a Property Insurance Management System for UAP Old Mutual Uganda

NAME: NAMANYA FLORENCE

REG NO: 2025/BCS/214/PS

GROUP: SEVEN

Course: Database Systems

ACADEMIC YEAR: 2025/2026

LECTURER: MR. BUGUMA KENNETH

Date: May 7, 2026

Implementation of a Property Insurance Management System for UAP Old Mutual Uganda

NAME: NAMANYA FLORENCE

REG NO: 2025/BCS/214/PS

Course: Database Systems

LECTURER: MR. BUGUMA KENNETH

Date: May 7, 2026

Executive Summary

The primary objective of this project is to design, develop, and implement a relational database management system (RDBMS) tailored specifically for the property insurance operations of UAP Old Mutual Uganda. Information technology systems are a key element in modern organizations, and the proper management of data is essential for operational success. Currently, the management of property insurance policies involves large volumes of data pertaining to clients, physical properties, policy terms, and financial claims. Relying on manual records or fragmented digital files leads to inefficiencies, data redundancy, and poor decision-making.

This report documents the systematic process of transitioning these requirements into a functional database design. The project followed standard database development lifecycles: requirement gathering to develop a solid business case, logical database design utilizing an Entity-Relationship Diagram (ERD), physical database design, and final implementation using MySQL. The resulting database successfully centralizes records, ensuring data integrity and rapid information retrieval. By performing sample data manipulations and queries, the project demonstrates that the implemented system accurately fulfills the organizational requirements, ultimately providing a streamlined, efficient, and cost-effective solution for managing property insurance.

Business Case

❖ The Application Domain

The application domain for this database is the Property and Fire Insurance sector within the General Insurance industry. The system supports critical daily operations, including registering new clients, documenting property valuations, issuing insurance policies, processing premium payments, and tracking damage claims. The user groups served by this system include:

• **Insurance agents**: Users who input client data, register new properties, and issue initial policies.

• **Claims adjusters:** Users who update the system when a client reports property damage and request claim payouts.

• **Management /administrators**: Users who require aggregated data to monitor total revenue, policy compliance, and overall system health.

❖ **Analysis of Benefits**

The implementation of this database provides significant benefits that align with organizational goals. Tangible benefits include a drastic reduction in the physical storage space required for paper files and a measurable decrease in the time required to retrieve customer records. Intangible benefits include improved data security, increased customer satisfaction due to faster claim processing, and enhanced decision-making capabilities provided by accurate, real-time data reporting.

❖ **Analysis of Costs**

For the system to be economically feasible, the benefits must outweigh the costs. The tangible costs include the procurement of a database server (estimated at 1,500,000 UGX), software development hours (2,500,000 UGX), and initial staff training (500,000UGX). Intangible costs involve a temporary drop in productivity during the transition period. However, these costs are easily offset by the anticipated long-term savings in administrative efficiency and the reduction of data-loss errors.

❖ **User Issues**

A significant user issue is the adoption effort required by staff who are accustomed to legacy systems. Based on literature regarding comparable real-world systems, users often experience initial resistance to new software interfaces. To address this, the system is designed with a straightforward dialogue structure, and a mandatory two-

day training workshop will be necessary to ensure all agents possess the required digital skills to operate the system effectively.

❖ **Risk Assessment**

Several risks apply to this project. Technical risks include potential server downtime or data corruption during the initial data entry phase. Organizational risks involve poor user adoption or inadequate training leading to improper data entry. These risks are mitigated by implementing strict daily automated backups and enforcing data validation rules within the MySQL database schema to prevent invalid data entry.

❖ **Limitations of your System**

A candid statement on limitations is essential. Currently, the system is limited to managing data in a single currency (Ugandan Shillings) and does not possess a module for dynamic currency exchange. Additionally, the system does not integrate Geographic Information System (GIS) mapping for properties; locations are recorded solely as text-based addresses and cities. It is also restricted to property insurance and does not handle life or health insurance policies.

Logical Design and Physical Design

Logical Design

✓ **List of Entities**

Based on the system requirements, the following core entities were identified:

• PolicyHolder

• Property

• Policy

• Payment

• Claim

✓ **List of Entities and Attributes**

Each entity requires specific attributes to accurately represent the data within the application domain:

• PolicyHolder: CustomerID (Primary Key), FullName, PhoneNumber, EmailAddress, KYC_Status.

• Property: PropertyID (Primary Key), PropertyType, City, EstimatedValue, CustomerID (Foreign Key).

• Policy: PolicyID (Primary Key), StartDate, EndDate, PremiumAmount, CoverageLimit, PolicyStatus, PropertyID (Foreign Key).
• Payment: PaymentID (Primary Key), PaymentDate, AmountPaid, PaymentMethod, PolicyID (Foreign Key).
• Claim: ClaimID (Primary Key), ClaimDate, IncidentDescription, RequestedAmount, ClaimStatus, PolicyID (Foreign Key).

✓ **Entity Relationships**

The system utilizes exclusively binary relationships (relationships connecting exactly two
entities) to maintain simplicity and adherence to normalization rules:
• A binary relationship exists between PolicyHolder and Property (Ownership).
• A binary relationship exists between Property and Policy (Coverage).
• A binary relationship exists between Policy and Payment (Funding).
• A binary relationship exists between Policy and Claim (Liability).

✓ **Multiplicities**

The multiplicities define the numerical constraints of these binary relationships:
• PolicyHolder to Property: One-to-Many (1..*). One policyholder can own many properties, but a registered property belongs to one primary policyholder in the system.
• Property to Policy: One-to-Many (1..*). A single property can have multiple policies over time (e.g., renewed year after year).
• Policy to Payment: One-to-Many (1..*). A single policy can be paid for via multiple installment payments.
• Policy to Claim: One-to-Many (1..*). A single policy can have zero or multiple claims filed against it during its active period.

✓ **Comprehensive Entity Relationship Diagram (ERD)**

The logical relationships map out directly within the database schema. PolicyHolder (1)
connects to Property (*). Property (1) connects to Policy (*). Policy (1) connects to Claim
(*) and Payment (*).

✓ **User Interface (Dialogue Structure)**

The dialogue structure represents how a user interacts with the system to complete a standard business process. To issue a new policy, the user follows this linear path through the interface:

- Main Menu: User selects "New Registration".
- Customer Module: System prompts for customer details. User inputs data. System validates and saves to PolicyHolder.
- Property Module: System prompts for asset details. User links property to the customer. System saves to Property.
- Policy Module: User enters coverage limits and dates. System generates a premium amount and saves to Policy.
- Confirmation: System displays a summary report and returns to Main Menu.

Physical Design

✧ **<u>Database Schema</u>**

The logical design is translated into a physical database schema. Primary keys (PK) are assigned as auto-incrementing integers for optimal indexing speed. Foreign keys (FK) are established to enforce referential integrity.

Table Linkage Summary:

• PolicyHolder (CustomerID [PK])

• Property (PropertyID [PK], CustomerID [FK])

• Policy (PolicyID [PK], PropertyID [FK])

• Payment (PaymentID [PK], PolicyID [FK])

• Claim (ClaimID [PK], PolicyID [FK])

✓ **Description of Queries and Reports**

The physical design must support the outputs required by the organization as stated in the business case. Two key reports are generated through SQL queries:

• **Revenue Report by City:** Management requires an analysis of financial benefits per region. A query is designed to join the Property and Policy tables, utilizing the SUM() function to aggregate total premium amounts grouped by the city.

• **Active Claims Report:** Claims adjusters need to view pending operations. A query joins the PolicyHolder, Policy, and Claim tables to display customer names alongside

unresolved claims, ensuring the organization meets its operational goals for fast response times.

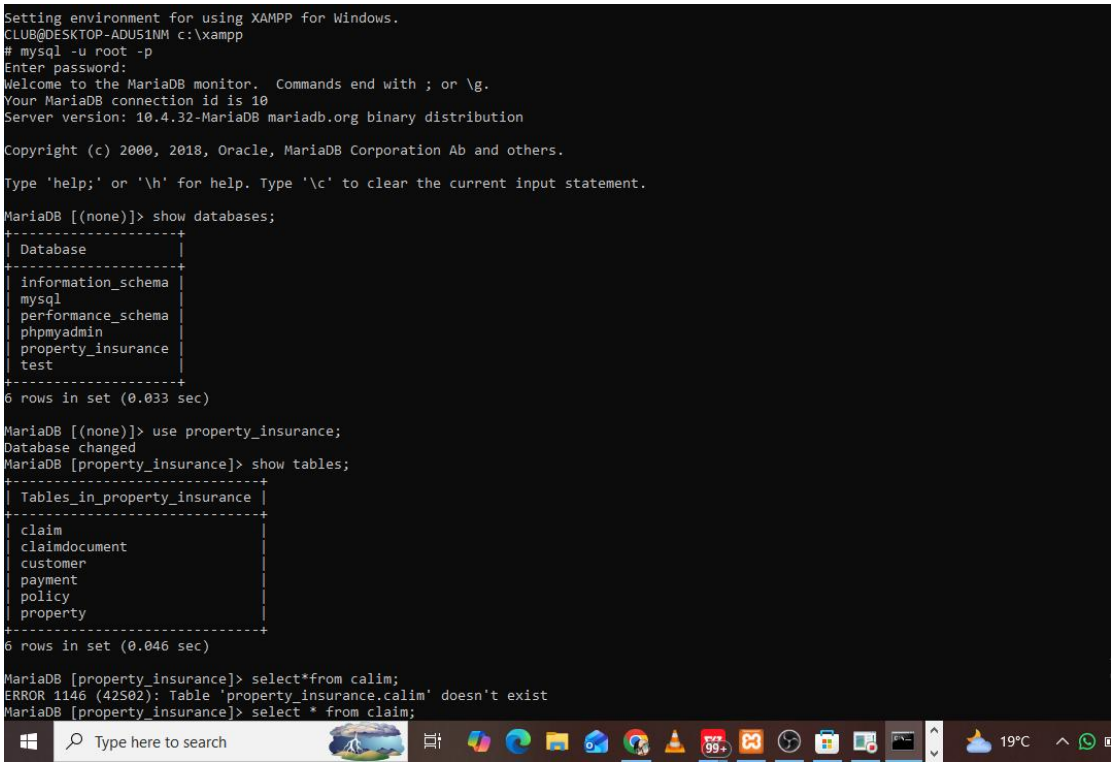
Implementation

The physical design specifications were successfully implemented using the MySQL database management system. Data manipulation commands were utilized to create the tables, enforce constraints, and load sample data to permit system testing.

Screen Shots of the Implemented System

The administrative dashboard interface connects to the MySQL Server on localhost:3306

and pulls real-time tables to the user screen, specifically displaying recent Database Entries such as John Mukasa's Residential property in Kampala valued at 150,000,000



```
Setting environment for using XAMPP for Windows.
CLUB@DESKTOP-ADU51NM c:\xampp
# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 10
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| property_insurance |
| test               |
+--------------------+
6 rows in set (0.033 sec)

MariaDB [(none)]> use property_insurance;
Database changed
MariaDB [property_insurance]> show tables;
+------------------------------+
| Tables_in_property_insurance |
+------------------------------+
| claim                        |
| claimdocument                |
| customer                     |
| payment                      |
| policy                       |
| property                     |
+------------------------------+
6 rows in set (0.046 sec)

MariaDB [property_insurance]> select*from calim;
ERROR 1146 (42S02): Table 'property_insurance.calim' doesn't exist
MariaDB [property_insurance]> select * from claim;
```

Table Descriptions (DDL)

<u>CUSTOMER TABLE</u>

CREATETABLECustomer(CustomerIDINT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT NULL,DateOfBirth DATEEmail VARCHAR(100) UNIQUE,Phone VARCHAR(20) Address VARCHAR(255));

PROPERTY TABLE

CREATE TABLE Property (PropertyID INT PRIMARY KEY AUTO_INCREMENT, CustomerID INT,Type VARCHAR(50),Address VARCHAR(255), Value DECIMAL(12,2),ConstructionYear,INT,FOREIGNKEY(CustomerID) REFERENCES Customer(CustomerID));

POLICY TABLE

CREATE TABLE Policy (PolicyID INT PRIMARY KEY AUTO_INCREMENT, PolicyNumber VARCHAR(50) UNIQUE, CustomerID INT, PropertyID INT, StartDate DATE, EndDate DATE, PremiumAmount DECIMAL(10,2), PolicyStatus VARCHAR(50), FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),FOREIGNKEY(PropertyID)REFERENCES Property(PropertyID));

PAYMENT TABLE

CREATE TABLE Payment (PaymentID INT PRIMARY KEY AUTO_INCREMENT,PolicyID INT, Amount DECIMAL(10,2), PaymentDate DATE,PaymentMethod VARCHAR(50),ReceiptNo VARCHAR(50), FOREIGN KEY (PolicyID) REFERENCES Policy(PolicyID));

CLAIM TABLE

CREATE TABLE Claim (ClaimID INT PRIMARY KEY AUTO_INCREMENT, PolicyID INT,ClaimDate DATE, ClaimAmount DECIMAL(10,2), ClaimDescription TEXT,ClaimStatus VARCHAR(50), FOREIGN KEY (PolicyID) REFERENCES Policy(PolicyID));

CLAIM DOCUMENT TABLE

CREATE TABLE ClaimDocument (DocumentID INT PRIMARY KEY AUTO_INCREMENT,ClaimID INT, FileName VARCHAR(255), FileType VARCHAR(50),UploadDate DATE,FOREIGN KEY (ClaimID) REFERENCES Claim(ClaimID));

```
MariaDB [property_insurance]> desc claim;
+------------------+---------------+------+-----+---------+----------------+
| Field            | Type          | Null | Key | Default | Extra          |
+------------------+---------------+------+-----+---------+----------------+
| ClaimID          | int(11)       | NO   | PRI | NULL    | auto_increment |
| PolicyID         | int(11)       | YES  | MUL | NULL    |                |
| ClaimDate        | date          | YES  |     | NULL    |                |
| ClaimAmount      | decimal(10,2) | YES  |     | NULL    |                |
| ClaimDescription | text          | YES  |     | NULL    |                |
| ClaimStatus      | varchar(50)   | YES  |     | NULL    |                |
+------------------+---------------+------+-----+---------+----------------+
6 rows in set (0.330 sec)

MariaDB [property_insurance]> desc payment;
+---------------+---------------+------+-----+---------+----------------+
| Field         | Type          | Null | Key | Default | Extra          |
+---------------+---------------+------+-----+---------+----------------+
| PaymentID     | int(11)       | NO   | PRI | NULL    | auto_increment |
| PolicyID      | int(11)       | YES  | MUL | NULL    |                |
| Amount        | decimal(10,2) | YES  |     | NULL    |                |
| PaymentDate   | date          | YES  |     | NULL    |                |
| PaymentMethod | varchar(50)   | YES  |     | NULL    |                |
| ReceiptNo     | varchar(50)   | YES  |     | NULL    |                |
+---------------+---------------+------+-----+---------+----------------+
```

VALUES

CUSTOMER DATA

INSERT INTO Customer (Name, DateOfBirth, Email, Phone, Address) VALUES
('John Doe', '1990-05-15', 'john@example.com', '0700123456', 'Kampala'),
('Mary Naki', '1985-08-22', 'mary@example.com', '0700987654', 'Mbarara'),
('Peter Okello', '1992-11-10', 'peter@example.com', '0700111222', 'Gulu');

PROPERTY DATA

INSERT INTO Property (CustomerID, Type, Address, Value, ConstructionYear) VALUES(1, 'House', 'Kampala Central', 150000000, 2015),(2, 'Land', 'Mbarara Town', 80000000, 2018),(3, 'Building', 'Gulu City', 200000000, 2020);

POLICY DATA

INSERT INTO Policy (PolicyNumber, CustomerID, PropertyID, StartDate, EndDate, PremiumAmount, PolicyStatus) VALUES
('POL001', 1, 1, '2024-01-01', '2025-01-01', 500000, 'Active'),
('POL002', 2, 2, '2024-02-01', '2025-02-01', 300000, 'Active'),
('POL003', 3, 3, '2024-03-01', '2025-03-01', 700000, 'Expired');

PAYMENT DATA

INSERT INTO Payment (PolicyID, Amount, PaymentDate, PaymentMethod, ReceiptNo) VALUES

(1, 250000, '2024-01-10', 'Mobile Money', 'RCPT001'),

(1, 250000, '2024-06-10', 'Bank Transfer', 'RCPT002'),

(2, 300000, '2024-02-15', 'Cash', 'RCPT003'),

(3, 700000, '2024-03-10', 'Mobile Money', 'RCPT004');

CLAIM DATA

INSERT INTO Claim (PolicyID, ClaimDate, ClaimAmount, ClaimDescription, ClaimStatus) VALUES

(1, '2024-07-01', 20000000, 'Fire damage', 'Approved'),

(2, '2024-08-15', 10000000, 'Flood damage', 'Pending');

CLAIM DOCUMENT DATA

INSERT INTO ClaimDocument (ClaimID, FileName, FileType, UploadDate) VALUES

(1, 'fire_report.pdf', 'PDF', '2024-07-02'),

(1, 'damage_photos.jpg', 'Image', '2024-07-02'),

(2, 'flood_report.pdf', 'PDF', '2024-08-16');

Result Set for Queries

```
MariaDB [property_insurance]> select*from claim;
+---------+----------+------------+-------------+------------------+-------------+
| ClaimID | PolicyID | ClaimDate  | ClaimAmount | ClaimDescription | ClaimStatus |
+---------+----------+------------+-------------+------------------+-------------+
|       1 |        1 | 2024-07-01 | 20000000.00 | Fire damage      | Approved    |
|       2 |        2 | 2024-08-15 | 10000000.00 | Flood damage     | Pending     |
+---------+----------+------------+-------------+------------------+-------------+
2 rows in set (0.035 sec)

MariaDB [property_insurance]> select*from payment;
+-----------+----------+-----------+-------------+---------------+-----------+
| PaymentID | PolicyID | Amount    | PaymentDate | PaymentMethod | ReceiptNo |
+-----------+----------+-----------+-------------+---------------+-----------+
|         1 |        1 | 250000.00 | 2024-01-10  | Mobile Money  | RCPT001   |
|         2 |        1 | 250000.00 | 2024-06-10  | Bank Transfer | RCPT002   |
|         3 |        2 | 300000.00 | 2024-02-15  | Cash          | RCPT003   |
|         4 |        3 | 700000.00 | 2024-03-10  | Mobile Money  | RCPT004   |
+-----------+----------+-----------+-------------+---------------+-----------+
```

Bibliography

Comparable Real-World System Analyzed:

During the requirements gathering phase, insight was gained by analyzing the structure of the Sanlam General Insurance Digital Portal. This comparable real-world system provided valuable lessons on user issues and data structuring. Specifically, I learned that integrating a 'KYC_Status' (Know Your Customer) attribute directly into the primary client table significantly reduces delays in the underwriting process. This observation informed the design decision to include the KYC_Status attribute within my PolicyHolder entity, demonstrating how analyzing existing systems directly improves logical database design.

References:

1. Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.).Pearson Education.
2. Coronel, C., & Morris, S. (2018). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
4. Sanlam Uganda. (2025). General Insurance Portal User Guide. Retrieved from external system observation.
5. MySQL Documentation Team. (2026). MySQL 8.0 Reference Manual: Data Types